

**КРОСПЛАТФОРМНА КЛІЄНТ-СЕРВЕРНА СИСТЕМА АГРЕГАЦІЇ ТА ОБРОБКИ
МЕДИКО-БІОЛОГІЧНИХ ДАНИХ З ВИКОРИСТАННЯМ ХМАРНИХ ТЕХНОЛОГІЙ**

Опис програми

ІАЛЦ.467200.004 ОП

jstp.js

lib/connection.js

lib/net.js

lib/server.js

lib/socket.js

lib/stringify.js

lib/transport-common.js

src/common.h

#ifndef SRC_COMMON_H_

#define SRC_COMMON_H_

#define THROW_EXCEPTION(ex_type, ex_msg)

isolate->ThrowException(v8::Exception::ex_type(

v8::String::NewFromUtf8(isolate, ex_msg)))

#endif // SRC_COMMON_H_

src/message_parser.h

#ifndef SRC_MESSAGE_PARSER_H_

#define SRC_MESSAGE_PARSER_H_

#include <v8.h>

namespace jstp {

					ІАЛЦ.467200.004 ОП			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Льоскін І. В.				Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Опис програми	Лит.	Аркуш	Аркушів
Перевірів	Кобилюк А. Г.						2	13
Н. контр.	Сімоненко В. П.					НТУУ "КПІ ім. Ізгоря Сікорського", ФІОТ ІО-25		
Затвердив	Волокита А. М.							

```

namespace message_parser {

// Efficiently parses JSTP messages for transports that require message
// delimiters eliminating the need to split the stream data into parts before
// parsing and allowing to do that in one pass.
v8::Local<v8::String> ParseNetworkMessages(v8::Isolate* isolate,
    const v8::String::Utf8Value& in, v8::Local<v8::Array> out);

} // namespace message_parser

} // namespace jstp

#endif // SRC_MESSAGE_PARSER_H_

src/message_parser.cc

#include "message_parser.h"

#include <cstddef>
#include <cstring>

#include <v8.h>

#include "common.h"
#include "parser.h"

using std::size_t;
using std::strlen;

```

```

using v8::Array;
using v8::Isolate;
using v8::Local;
using v8::String;

using jstp::parser::internal::ParseObject;
using jstp::parser::internal::SkipToNextToken;

namespace jstp {

namespace message_parser {

Local<String> ParseNetworkMessages(Isolate* isolate,
                                   const String::Utf8Value& in,
                                   Local<Array> out) {

    const size_t total_size = in.length();
    size_t parsed_size = 0;
    const char* source = *in;
    const char* curr_chunk = source;
    int index = 0;

    while (parsed_size < total_size) {
        auto chunk_size = strlen(curr_chunk);
        parsed_size += chunk_size + 1;

        if (parsed_size <= total_size) {
            size_t skipped_size = SkipToNextToken(curr_chunk,
                                                    curr_chunk + chunk_size);

```

```

size_t parsed_chunk_size;
auto result = ParseObject(isolate, curr_chunk + skipped_size,
    curr_chunk + chunk_size, &parsed_chunk_size);

if (result.IsEmpty()) {
    return Local<String>();
}

parsed_chunk_size += skipped_size;
parsed_chunk_size += SkipToNextToken(curr_chunk + parsed_chunk_size,
    curr_chunk + chunk_size);

if (parsed_chunk_size != chunk_size) {
    THROW_EXCEPTION(SyntaxError, "Invalid format");
    return Local<String>();
}

out->Set(index++, result.ToLocalChecked());
curr_chunk += chunk_size + 1;
}
}

auto rest = String::NewFromUtf8(isolate, curr_chunk);
return rest;
}

} // namespace message_parser

```

```
} // namespace jstp
```

src/parser.h

```
#ifndef SRC_PARSER_H_
```

```
#define SRC_PARSER_H_
```

```
#include <cstdint>
```

```
#include <v8.h>
```

```
namespace jstp {
```

```
namespace parser {
```

```
// Deserializes a UTF-8 encoded string into a JavaScript value
```

```
// and returns a handle to it.
```

```
v8::Local<v8::Value> Parse(v8::Isolate* isolate,  
                           const v8::String::Utf8Value& in);
```

```
namespace internal {
```

```
// Returns count of bytes needed to skip to next token.
```

```
size_t SkipToNextToken(const char* str, const char* end);
```

```
// Parses an undefined value from 'begin' but never past 'end' and returns the
```

```
// parsed JavaScript value. The 'size' is incremented by the number of
```

```
// characters the function has used in the string so that the calling side
```

```
// knows where to continue from.
```

					ІА/Ц.467200.004 ОП	Аркуш
						6
Зм.	Лист	№ докум.	Підп.	Дата		

```

v8::MaybeLocal<v8::Value> ParseUndefined(v8::Isolate* isolate,
                                           const char* begin,
                                           const char* end,
                                           std::size_t* size);

```

// Parses a null value from ‘begin’ but never past ‘end’ and returns the parsed
// JavaScript value. The ‘size’ is incremented by the number of characters the
// function has used in the string so that the calling side knows where to
// continue from.

```

v8::MaybeLocal<v8::Value> ParseNull(v8::Isolate* isolate,
                                      const char* begin,
                                      const char* end,
                                      std::size_t* size);

```

// Parses a boolean value from ‘begin’ but never past ‘end’ and returns the
// parsed JavaScript value. The ‘size’ is incremented by the number of
// characters the function has used in the string so that the calling side
// knows where to continue from.

```

v8::MaybeLocal<v8::Value> ParseBool(v8::Isolate* isolate,
                                      const char* begin,
                                      const char* end,
                                      std::size_t* size);

```

// Parses a numeric value from ‘begin’ but never past ‘end’ and returns the
// parsed JavaScript value. The ‘size’ is incremented by the number of
// characters the function has used in the string so that the calling side
// knows where to continue from.

```

v8::MaybeLocal<v8::Value> ParseNumber(v8::Isolate* isolate,

```

```

        const char* begin,
        const char* end,
        std::size_t* size);

// Parses a string value from 'begin' but never past 'end' and returns the
// parsed JavaScript value. The 'size' is incremented by the number of
// characters the function has used in the string so that the calling side
// knows where to continue from.
v8::MaybeLocal<v8::Value> ParseString(v8::Isolate* isolate,
        const char* begin,
        const char* end,
        std::size_t* size);

// Parses an array from 'begin' but never past 'end' and returns the parsed
// JavaScript value. The 'size' is incremented by the number of characters the
// function has used in the string so that the calling side knows where to
// continue from.
v8::MaybeLocal<v8::Value> ParseArray(v8::Isolate* isolate,
        const char* begin,
        const char* end,
        std::size_t* size);

// Parses an object key from 'begin' but never past 'end' and returns
// the parsed JavaScript value. The 'size' is incremented by the number
// of characters the function has used in the string so that the calling side
// knows where to continue from.
v8::MaybeLocal<v8::String> ParseKeyInObject(v8::Isolate* isolate,
        const char* begin,

```



```

const char* end,
std::size_t* size);

// Parses a value corresponding to key inside object from ‘begin‘
// but never past ‘end‘ and returns the parsed JavaScript value.
// The ‘size‘ is incremented by the number of characters the function has used
// in the string so that the calling side knows where to continue from.
v8::MaybeLocal<v8::Value> ParseValueInObject(v8::Isolate* isolate,
const char* begin,
const char* end,
std::size_t* size);

// Parses an object from ‘begin‘ but never past ‘end‘ and returns the parsed
// JavaScript value. The ‘size‘ is incremented by the number of characters the
// function has used in the string so that the calling side knows where to
// continue from.
v8::MaybeLocal<v8::Value> ParseObject(v8::Isolate* isolate,
const char* begin,
const char* end,
std::size_t* size);

// Parses a decimal number, either integer or float.
v8::MaybeLocal<v8::Value> ParseDecimalNumber(v8::Isolate* isolate,
const char* begin,
const char* end,
std::size_t* size,
bool negate_result);

```

```

// Parses an integer number in arbitrary base without prefixes.
v8::Local<v8::Value> ParseIntegerNumber(v8::Isolate* isolate,
                                         const char* begin,
                                         const char* end,
                                         std::size_t* size,
                                         int      base,
                                         bool      negate_result);

// Parses an integer number, which is too big to be parsed using
// ParseIntegerNumber, in arbitrary base without prefixes.
v8::Local<v8::Value> ParseBigIntegerNumber(v8::Isolate* isolate,
                                           const char* begin,
                                           const char* end,
                                           std::size_t* size,
                                           int      base,
                                           bool      negate_result);

} // namespace internal

} // namespace parser

} // namespace jstp

#endif // SRC_PARSER_H_

src/node_bindings.cc

#include <node.h>
#include <v8.h>

```

```

#include "common.h"
#include "parser.h"
#include "message_parser.h"

using v8::Array;
using v8::FunctionCallbackInfo;
using v8::HandleScope;
using v8::Isolate;
using v8::Local;
using v8::Object;
using v8::String;
using v8::Value;

namespace jstp {

namespace bindings {

void Parse(const FunctionCallbackInfo<Value>& args) {
    Isolate* isolate = args.GetIsolate();

    if (args.Length() != 1) {
        THROW_EXCEPTION(TypeError, "Wrong number of arguments");
        return;
    }

    if (!args[0]->IsString() && !args[0]->IsUint8Array()) {
        THROW_EXCEPTION(TypeError, "Wrong argument type");
        return;
    }
}
}
}

```

```
}
```

```
HandleScope scope(isolate);
```

```
String::Utf8Value str(args[0]->ToString());
```

```
auto result = jstp::parser::Parse(isolate, str);
```

```
args.GetReturnValue().Set(result);
```

```
}
```

```
void ParseNetworkMessages(const FunctionCallbackInfo<Value>& args) {
```

```
Isolate* isolate = args.GetIsolate();
```

```
if (args.Length() != 2) {
```

```
    THROW_EXCEPTION(TypeError, "Wrong number of arguments");
```

```
    return;
```

```
}
```

```
if (!args[0]->IsString() || !args[1]->IsArray()) {
```

```
    THROW_EXCEPTION(TypeError, "Wrong argument type");
```

```
    return;
```

```
}
```

```
HandleScope scope(isolate);
```

```
String::Utf8Value str(args[0]->ToString());
```

```
auto array = args[1].As<Array>();
```

```
auto result = jstp::message_parser::ParseNetworkMessages(isolate, str, array);
```

```
args.GetReturnValue().Set(result);
```

```
}
```

					ІА/Ц.467200.004 ОП	Архив
Зм.	Лист	№ докум.	Підп.	Дата		12

```

void Init(Local<Object> target) {
    NODE_SET_METHOD(target, "parse", Parse);
    NODE_SET_METHOD(target, "parseNetworkMessages",
        ParseNetworkMessages);
}

NODE_MODULE(jstp, Init);

} // namespace bindings

} // namespace jstp

```

					ІАЛЦ.467200.004 ОП	Аркуш
						13
Зм.	Лист	№ докум.	Підп.	Дата		